

Lab 2 report

实验目的与内容

本次实验复习了 Verilog 语法，实现 CPU 的功能部件。

任务 1：寄存器堆设计

思路及代码：

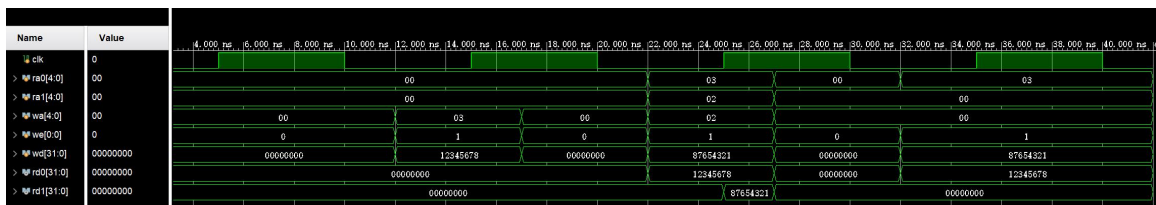
如图，首先进行初始化。由于读写异步，直接给 `rf_rd0` 和 `rf_rf1` 赋值。在时钟边沿到来时，在写使能信号有效的前提下，若写入的不是 0 号寄存器，则将 `rd_wd` 写入对应寄存器。这样 0 号寄存器保持值为 0 不变。

```
reg [31 : 0] reg_file [0 : 31];
// 用于初始化寄存器
integer i;
initial begin
    for (i = 0; i < 32; i = i + 1)
        reg_file[i] = 0;
end

assign rf_rd0 = reg_file [rf_ra0];
assign rf_rd1 = reg_file [rf_ra1];

always @(posedge clk) begin
    if(rf_we) begin
        if(rf_wa != 1'b0)
            reg_file[rf_wa] <= rf_wd;
    end
end
```

仿真：



任务 2: ALU 设计

思路及代码：直接对 alu_op 进行分类讨论即可。

```
wire slt_res, sltu_res;

SLT slt(alu_src0, alu_src1, slt_res);
SLTU sltu(alu_src0, alu_src1, sltu_res);

always @(*) begin
    case(alu_op)
        `ADD :
            alu_res = alu_src0 + alu_src1;
        `SUB :
            alu_res = alu_src0 - alu_src1;
        `SLT :
            alu_res = {{31{1'b0}}}, slt_res;
        `SLTU :
            alu_res = {{31{1'b0}}}, sltu_res;
        `AND :
            alu_res = alu_src0 & alu_src1;
        `OR :
            alu_res = alu_src0 | alu_src1;
        `XOR :
            alu_res = alu_src0 ^ alu_src1;
        `SLL :
            alu_res = alu_src0 << alu_src1;
        `SRL :
            alu_res = alu_src0 >> alu_src1;
        `SRA :
            alu_res = alu_src0 >>> alu_src1;
        `SRC0 :
            alu_res = alu_src0;
        `SRC1 :
            alu_res = alu_src1;
        default :
            alu_res = 32'H0;
    endcase
end
```

其中，slt_res 和 sltu_res 来自例化的 slt 和 sltu 模块。

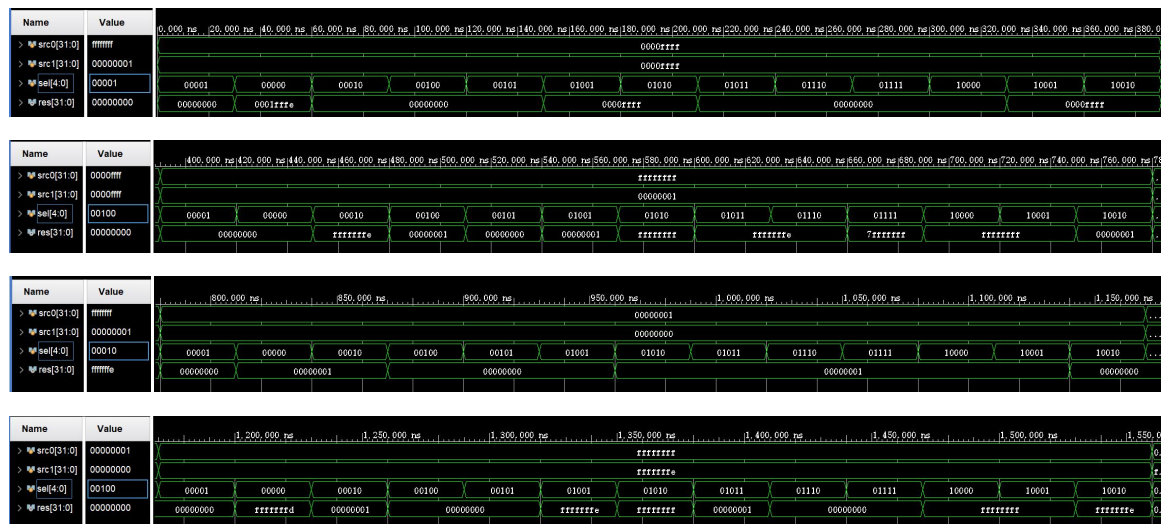
slt 模块：考虑讨论 a-b 的正负性，并对其是否溢出进行讨论。

```
module SLT(  
    input [31:0] a, b,  
    output lt  
);  
  
wire[31:0] outs = a - b;  
  
wire ovfs = ( ~( a[31] ^ (~b[31]) ) ) & ( (~b[31]) ^ outs[31] );  
assign lt = ( a[31] & (~b[31]) ) | ( (~ovfs) & outs[31] );  
endmodule
```

sltu 模块：对后 31 位进行有符号比较，再结合 a[31]和 b[31]判断。

```
module SLTU(  
    input [31:0] a, b,  
    output ltu  
);  
  
wire lt;  
SLT slt(  
    .a({1'b0,a[30:0]}),  
    .b({1'b0,b[30:0]}),  
    .lt(lt)  
);  
  
assign ltu = (a[31] ^ b[31]) ? ~a[31] : lt;  
  
endmodule
```

仿真：



Name	Value	
> src0[31:0]	00000001	00000000
> src1[31:0]	00000000	00000000
> op[4:0]	00100	00001 00000 00010 00100 00101 01001 01010 01011 01110 01111 10000 10001 10010
> res[31:0]	00000000	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000

任务 3: ALU 设计

代码及思路：按照数据通路连线即可。in_extended 为符号拓展后的 in 信号。

```

reg [31:0] alu_src0, alu_src1, output_data;
reg [4:0] alu_op;
wire [31:0] alu_res, in_extended;

assign in_extended[31:4] = {28{in[4]}};
assign in_extended[3:0] = in[3:0];

ALU alu(alu_src0, alu_src1, alu_op, alu_res);

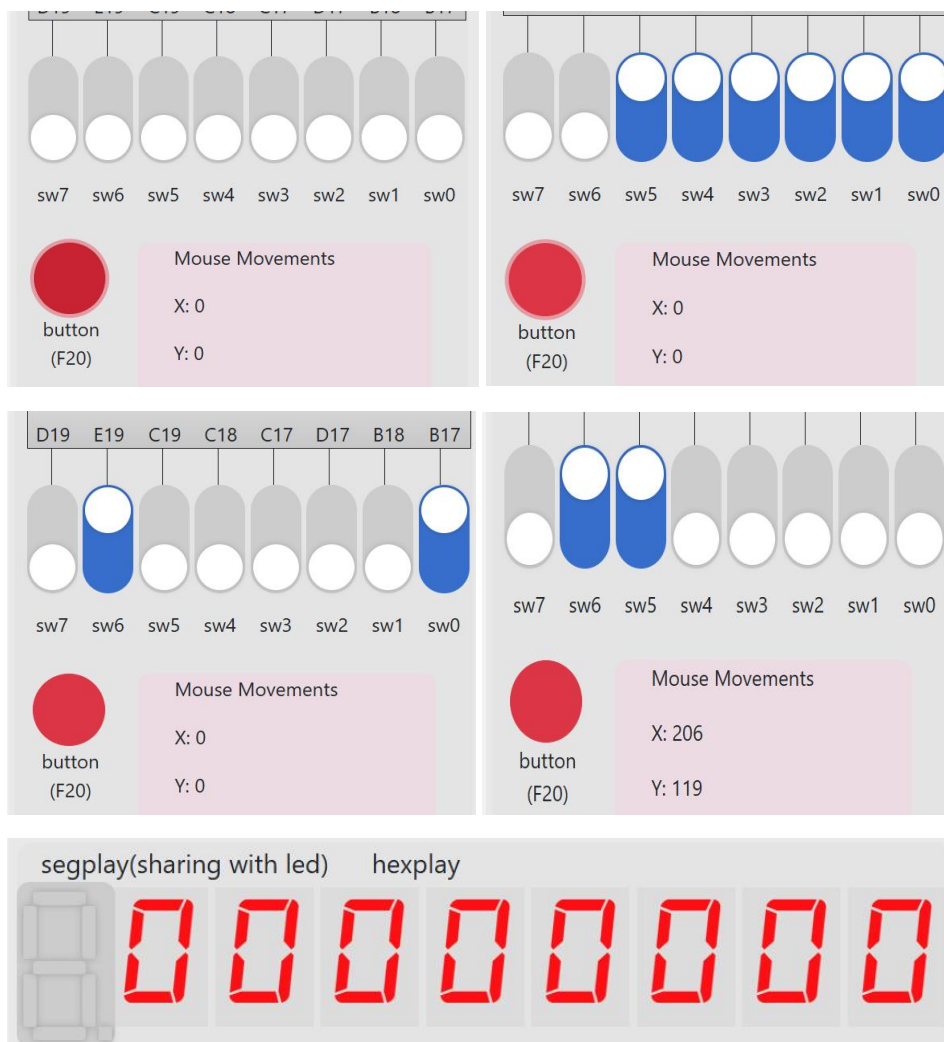
Segment segment(clk, rst, output_data, seg_data, seg_an);

always @(posedge clk) begin
    if(rst)begin
        alu_src0 <= 0;
        alu_src1 <= 0;
        alu_op <= 0;
        output_data <= 0;
    end else begin
        if(enable)begin
            case(ctrl)
                2'b00: alu_op <= in;
                2'b01: alu_src0 <= in_extended;
                2'b10: alu_src1 <= in_extended;
                2'b11: output_data <= alu_res;
            endcase
        end
    end
end

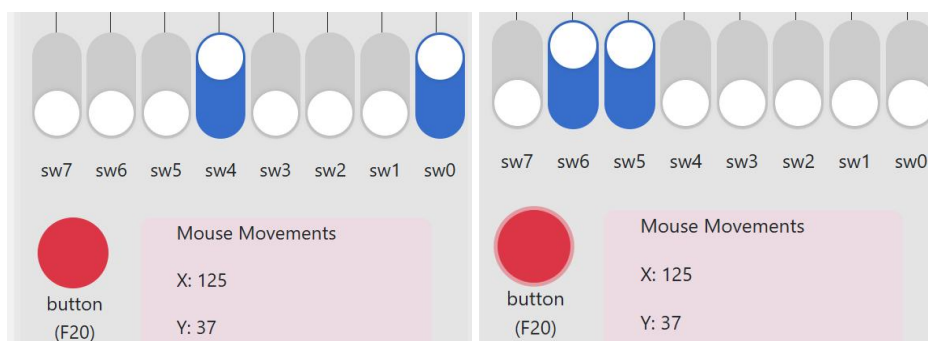
```

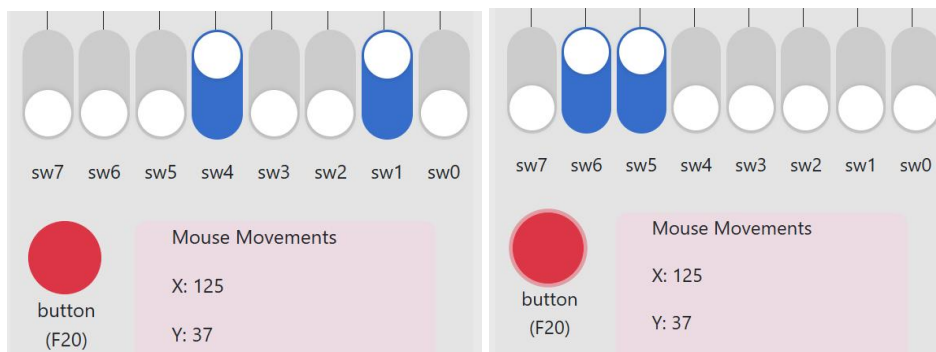
上板：

如图，如图依次操作，符号扩展后为 $0xFFFFFFFF + 0x00000001 = 0x00000000$

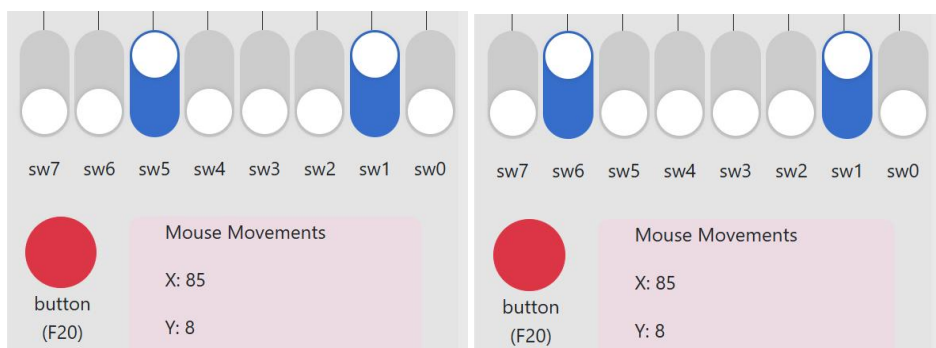


现在查看 sr0 和 sr1 的值:





自由就是拥有说二加二等于四的自由。现在再测试一下 $2+2=4$:



任务 4：初始化存储器

Load COE File

The initial memory content can be set by using a COE file. This will be passed to the core as a Memory Initialisation File (MIF).

Coefficients File



Options

Depth



[16 - 65536]

Data Width



[1 - 1024]

Memory Type

Memory Type



ROM



Single Port RAM



Simple Dual Port RAM



Dual Port RAM

Select th

Design Sources (2)



dist_mem_gen_0 (dist_mem_gen_0.xci) (1)

总结

本次实验复习了 verilog 语法，实现了 CPU 的功能部件，受益匪浅。